



JDBC进阶+Tomcat+初识Servlet

- 1 JDBC的SQL注入漏洞
- 2 Eclipse中配置Tomcat
- 3 初识Servlet

1

JDBC的SQL注入漏洞

2

Eclipse中配置Tomcat

3

初识Servlet

□ 什么是SQL注入漏洞？

- 一般而言，用户注册以后根据用户名和密码完成登录
- 注入漏洞：只需要知道用户名，即使密码不正确，也可以登录

□ 演示SQL注入漏洞？

- 在数据库中新建一张User表，有“ID”，“Username”，“Password”三个属性；
- 在表中添加一些数据；
- 修改原先的代码

JDBC的SQL注入漏洞

```
//用户登录
public static boolean login(String username, String password) throws Exception
{
    // TODO Auto-generated method stub
    Connection conn = null;
    Statement statement = null;
    ResultSet rs = null;
    boolean flag = false;

    // 1.加载驱动
    Class.forName("com.mysql.jdbc.Driver");
    // DriverManager.registerDriver(driver);
    // 2.获得连接
    conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/test_db", "root", "123456");
    //连接字符串, "test_db"数据库名称, "root"账户, "密码"
```

```
// 3.基本操作: 执行SQL
// 3.1获得执行SQL语句的对象
statement = conn.createStatement();
// 3.2编写SQL语句:
```

```
String sql = "select * from user where Username='"+username+"' and Password='"+password+"'"; //表名
```

```
// 3.3执行SQL:
rs = statement.executeQuery(sql);
// 3.4遍历结果集:
while(rs.next()){
    System.out.print(rs.getInt("id")+" ");
    System.out.print(rs.getString("Username")+" ");
    System.out.println();
    flag = true;
}
return flag;
}
```

```
public static void main(String[] args) throws Exception{
```

```
String username = "zhangsan";
String password = "1234";
if(login(username, password))
```

```
{
    System.out.println("登录成功!");
}
else
{
    System.out.println("登录失败!");
}
}
```

数据表

Id	Username	Password
1	zhangsan	1234
2	lisi	2345
3	wangwu	3456

程序运行结果

1 zhangsan
登录成功!

JDBC的SQL注入漏洞

```
public static void main(String[] args) throws Exception{
```

```
    //String username = "zhangsan";  
    //String password = "1234";
```

```
    String username = "zhangsan' or '1=1";  
    String password = "ffff";
```

```
    if(login(username, password))  
    {  
        System.out.println("登录成功!");  
    }  
    else  
    {  
        System.out.println("登录失败!");  
    }  
}
```

显然用户名密码错误，但依然能够登录成功！

输入方式一：

用户名' or '1=1 密码随意

程序运行结果

```
1 zhangsan  
登录成功！
```

JDBC的SQL注入漏洞

```
public static void main(String[] args) throws Exception{
```

```
    //String username = "zhangsan";  
    //String password = "1234";
```

```
    String username = "zhangsan' -- ";
```

```
    String password = "ffff";
```

```
    if(Login(username, password))
```

```
    {  
        System.out.println("登录成功!");
```

```
    }  
    else
```

```
    {  
        System.out.println("登录失败!");
```

```
    }  
}
```

显然用户名密码错误，但依然能够登录成功！

用户名' -- 密码随意

程序运行结果

```
1 zhangsan  
登录成功！
```

JDBC的SQL注入漏洞

□ JDBC的SQL注入漏洞原因分析

SQL注入漏洞分析

// 编写SQL语句:

```
String sql = "select * from user where username = '"+username+"' and password = '"+password+"'";
```

输入的用户名为:

* aaa' or '1=1' 密码随意

* aaa' -- 密码随意

原因:

* 在变量中存在SQL的关键字

* or

* --

这些都是SQL中的关键字:

```
select * from user where username = '变量' and password = '变量';
```

```
* select * from user where username = 'aaa' or '1=1' and password = ' ';
```

```
* select * from user where username = 'aaa' -- ' and password = ' ';
```

注: “--” 在SQL语句中是注释!!

□ JDBC的SQL注入漏洞解决

- 使用PreparedStatement对象
- 该对象会将SQL语句预先进行编译，使用?作为占位符，把SQL语句的格式固化；
- ?所代表内容是SQL语句中要传入的变量值；
- SQL的关键字将不会再被识别。

```
// 3.基本操作：执行SQL
// 3.1输入SQL语句
String sql = "select * from user where Username = ? and Password = ?";

// 3.2预编译SQL语句：
pstmt = conn.prepareStatement(sql);

// 3.3设置变量的值：
pstmt.setString(1, username);
pstmt.setString(2, password);

// 3.4进行查询
rs = pstmt.executeQuery();
```

- 1 JDBC的SQL注入漏洞
- 2 Eclipse中配置Tomcat
- 3 初识Servlet

□ Tomcat的目录结构

名称	修改日期	类型	大小
bin	2017/1/9 15:20	文件夹	
conf	2017/1/9 15:22	文件夹	
lib	2017/1/9 15:20	文件夹	
logs	2017/1/9 15:22	文件夹	
temp	2017/1/9 15:20	文件夹	
webapps	2017/1/9 15:20	文件夹	
work	2017/1/9 15:22	文件夹	

- bin: 二进制文件（命令文件：开启和关闭）
- conf: 配置文件
- lib: tomcat所需要的jar包
- logs: tomcat服务器日志文件
- temp: tomcat运行产生临时文件
- webapps: 需要发布的项目需要放在webapps下
- work: JSP翻译（编译）成Servlet产生的代码

□ 动态Web项目的目录结构

注：Tomcat服务器才能够处理动态的Web项目，Apache服务器只能处理静态的Web页面。

一个动态Web项目名称

- |-----静态页面（HTML、CSS、JS、图片）

- |-----JSP页面

- |-----WEB-INF

 - |-----web.xml（必须的）

 - |-----classes（Java类，可选的）

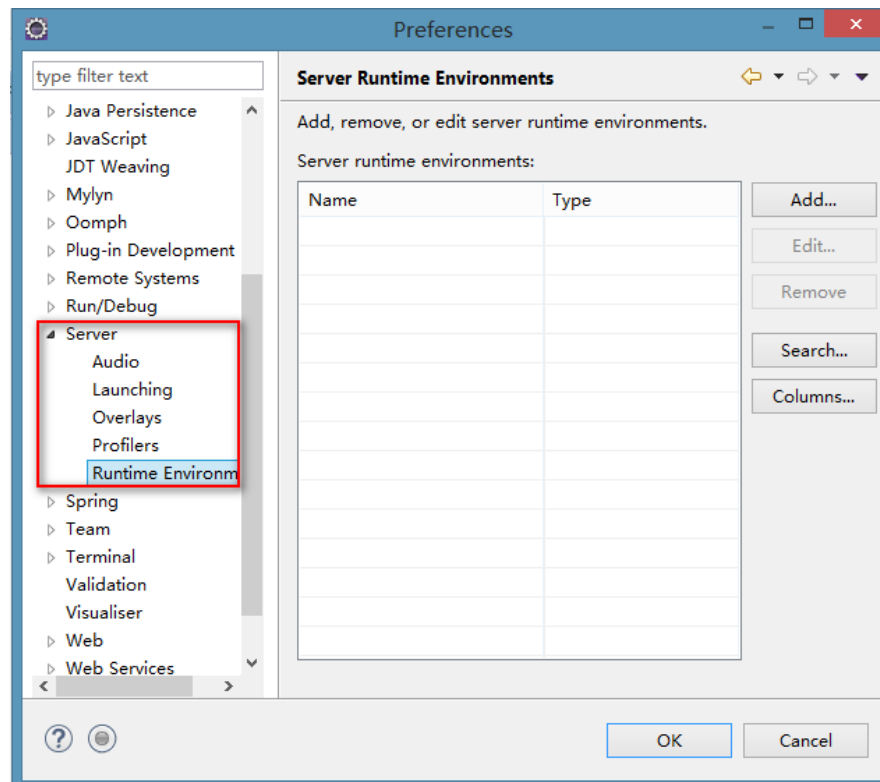
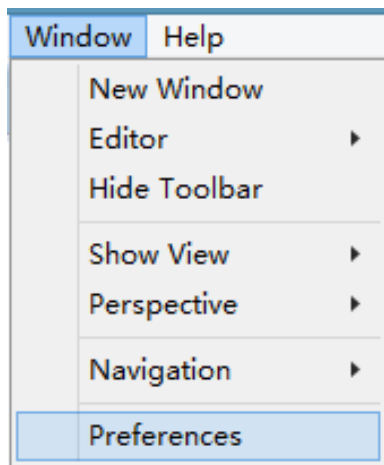
 - |-----lib （第三方的jar包，可选的，如连接数据库的jar包）

Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– Eclipse的服务器配置

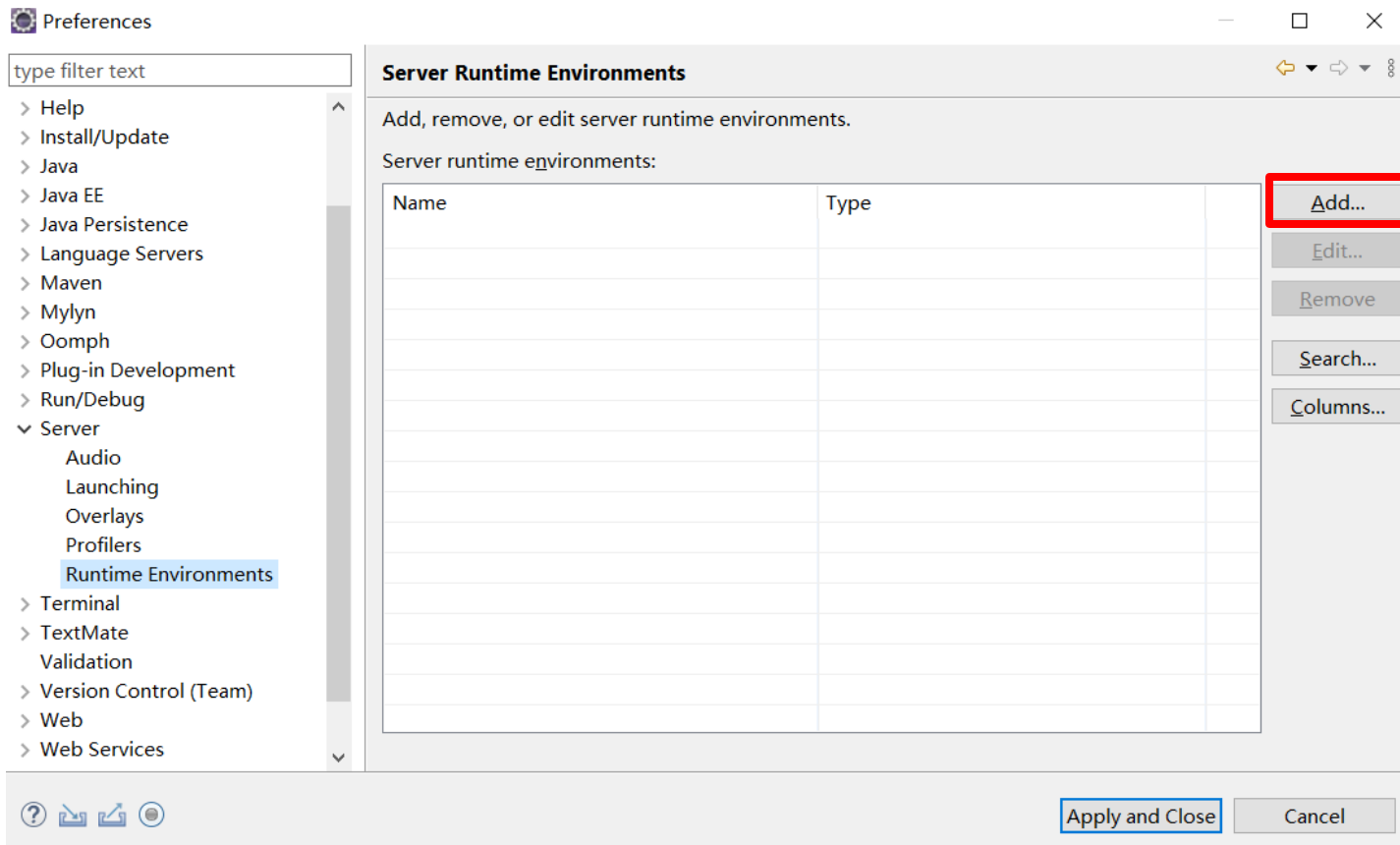
Window--Preference--Server--Runtime Environments



Eclipse中配置Tomcat

- ## ❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

Window--Preference--Server-- Runtime Environments

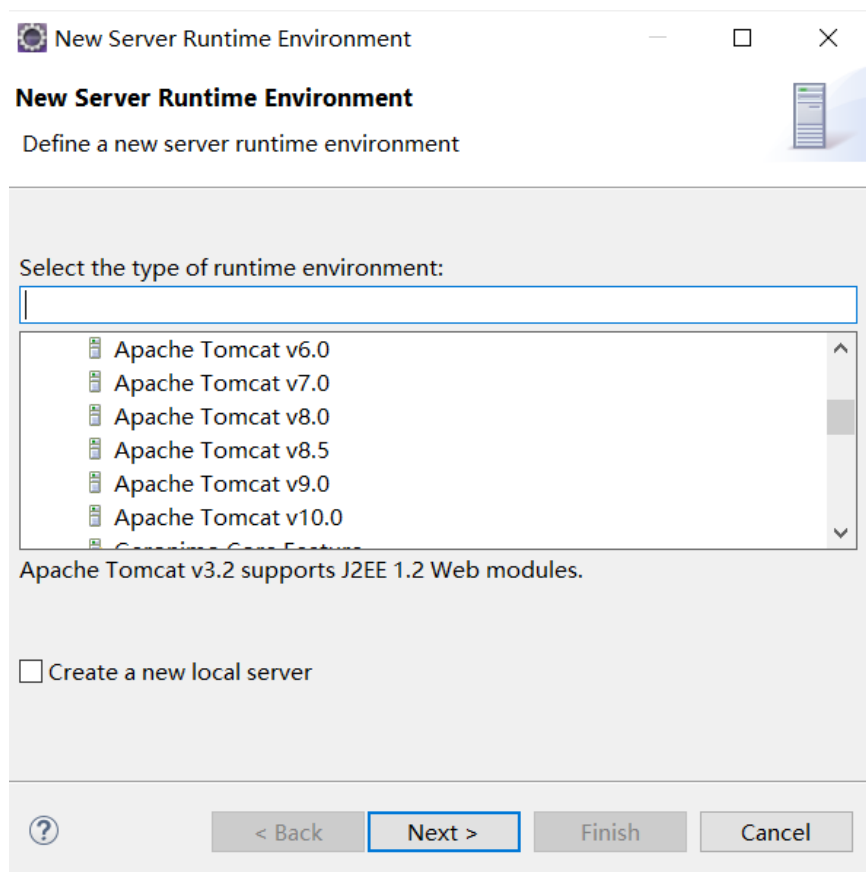


Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

— Eclipse的服务器配置

Window--Preference--Server-- Runtime Environments--Add



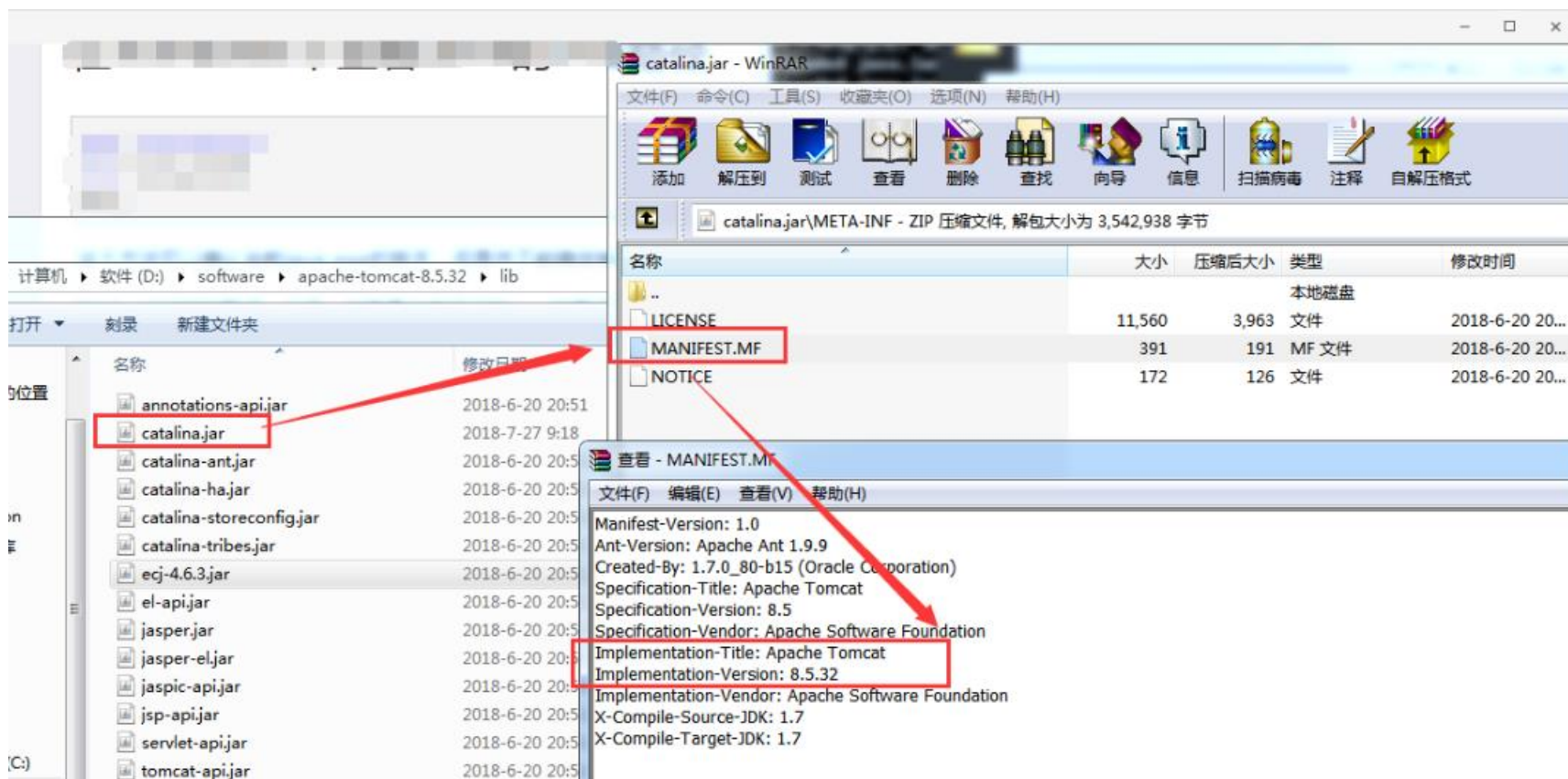
选择自己安装的
Tomcat服务器的版本

Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– 查看Tomcat的版本

tomcat/lib/catalina.jar，用解压工具打开，打开MANIFEST.MF

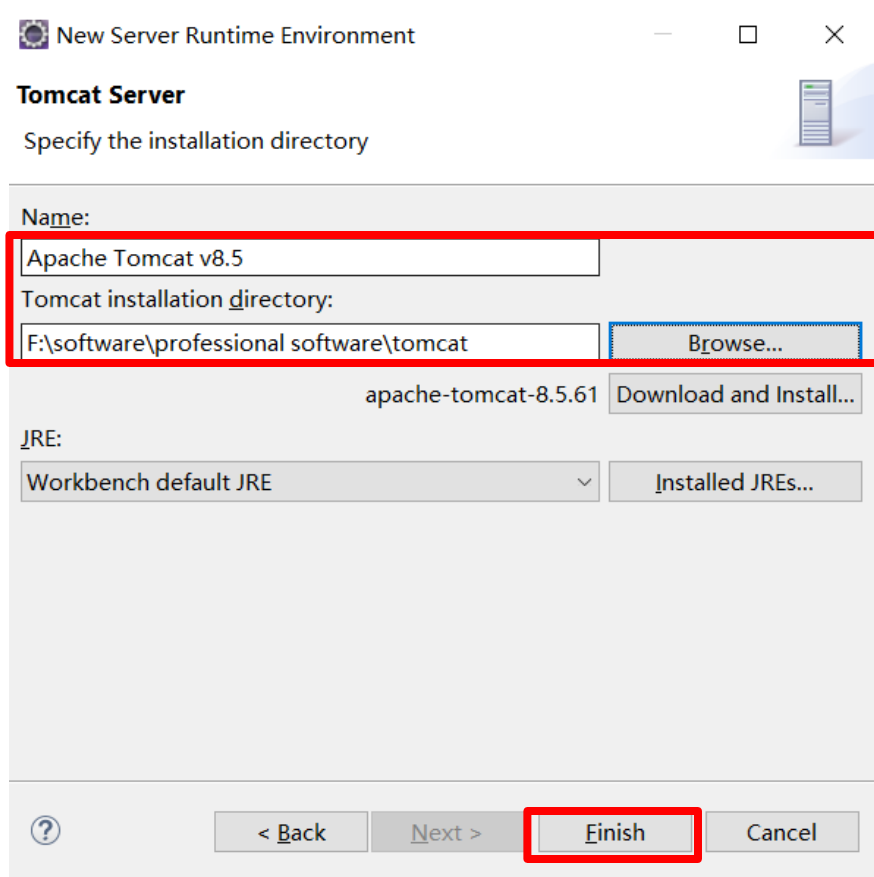


Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

— Eclipse的服务器配置

选择版本并添加路径--Finish

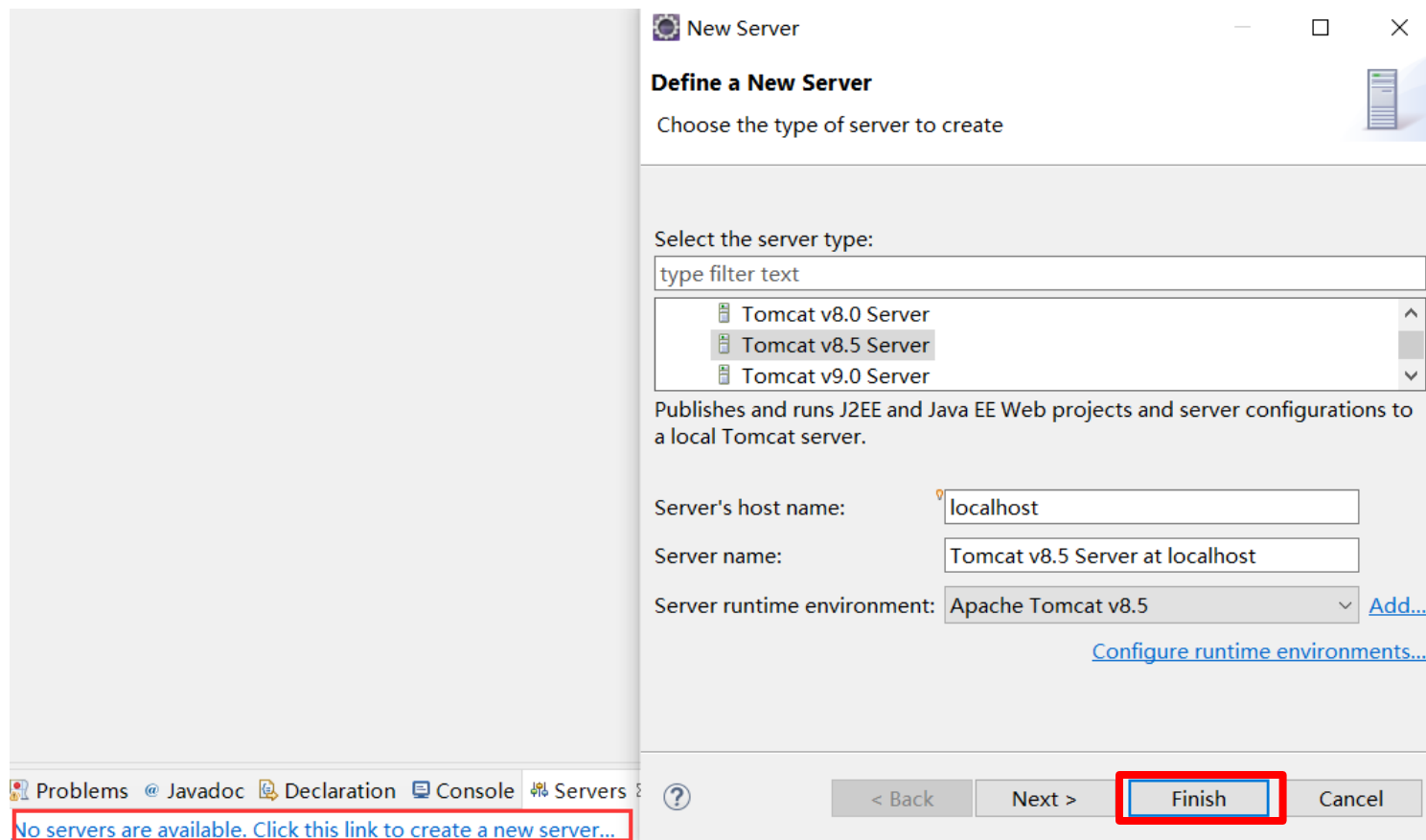


Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– Eclipse的服务器配置

打开Servers窗口--创建服务

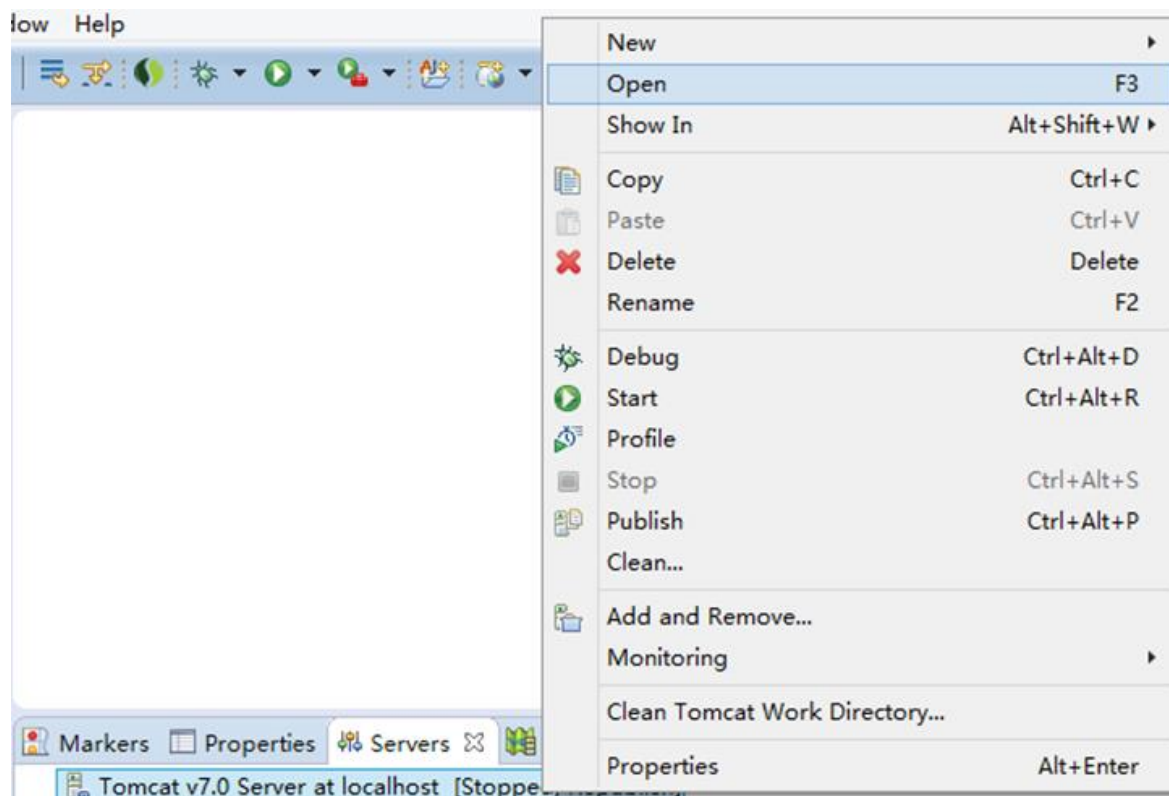


Eclipse中配置Tomcat

□ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– Eclipse的服务器配置

打开Servers窗口--创建服务--设置服务



右击Tomcat服务--Open

Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– Eclipse的服务器配置

打开Servers窗口--创建服务--设置服务

Overview

General Information

Specify the host name and other common settings.

Server name: Tomcat v8.5 Server at localhost
Host name: localhost
Runtime Environment: Apache Tomcat v8.5
Configuration path: /Servers/Tomcat v8.5 Server at localh Browse...
[Open launch configuration](#)

Server Locations

Specify the server path (i.e. catalina.base) and deploy path. Server must be published with no modules present to make changes.

- ☒ Use workspace metadata (does not modify Tomcat installation)
☐ Use Tomcat installation (takes control of Tomcat installation)
☐ Use custom location (does not modify Tomcat installation)

Server path: .metadata\.plugins\org.eclipse.wst.server.cor Browse...

[Set deploy path to the default value \(currently set\)](#)

Deploy path: wtpwebapps Browse...

Publishing

Timeouts

Ports

Modify the server ports.

Port Name	Port Number
Tomcat admin port	8005
HTTP/1.1	8080

MIME Mappings

Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

— Eclipse的服务器配置

打开Servers窗口--创建服务--设置服务

Overview

General Information

Specify the host name and other common settings.

Server name: Tomcat v8.5 Server at localhost

Host name: localhost

Runtime Environment: Apache Tomcat v8.5

Configuration path: /Servers/Tomcat v8.5 Server at localh Browse...

[Open launch configuration](#)

Server Locations

Specify the server path (i.e. catalina.base) and deploy path. Server must be published with no modules present to make changes.

☐ Use workspace metadata (does not modify Tomcat installation)

☒ Use Tomcat installation (takes control of Tomcat installation)

☐ Use custom location (does not modify Tomcat installation)

Server path: F:\software\professional software\tomcat Browse...

[Set deploy path to the default value](#)

Deploy path: webapps Browse...

Publishing

Timeouts

Ports

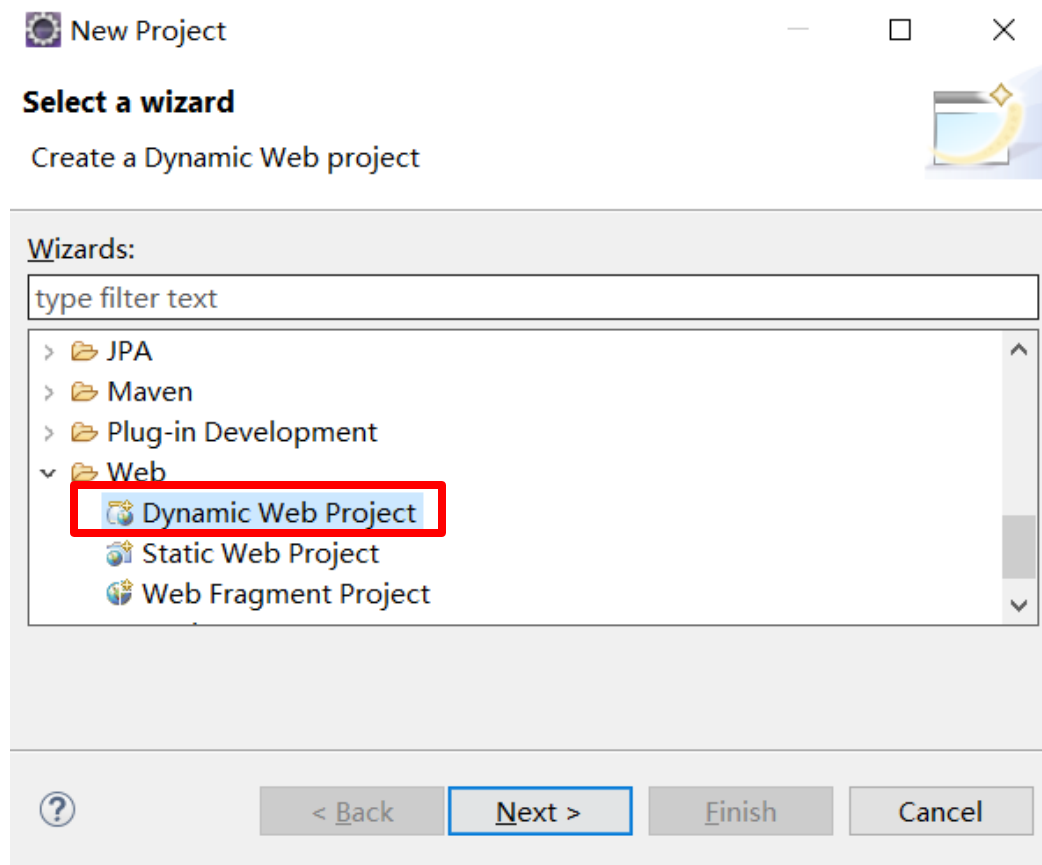
Modify the server ports.

Port Name	Port Number
Tomcat admin port	8005
HTTP/1.1	8080

MIME Mappings

Eclipse中配置Tomcat

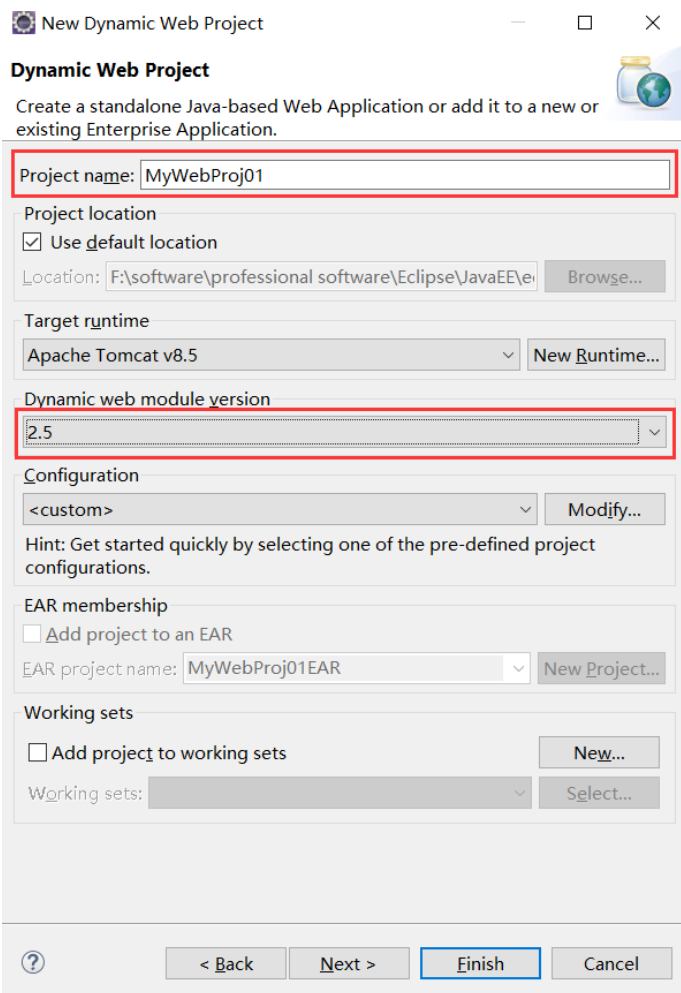
- 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中
 - 创建一个动态的Web项目（File--New--Project--Web—Dynamic Web Project）



Eclipse中配置Tomcat

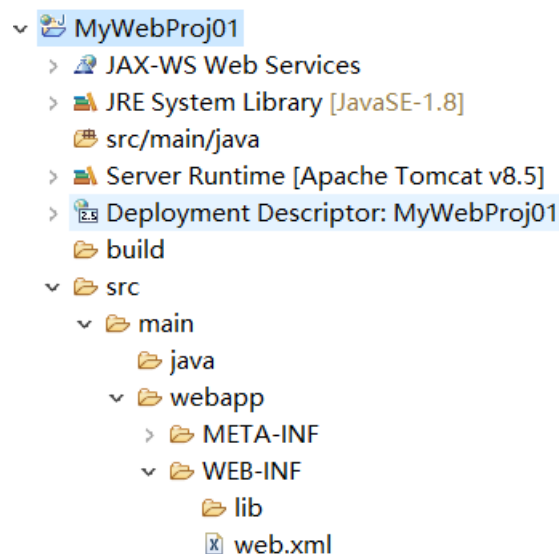
❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

— 创建一个动态的Web项目（File--New--Project--Web—Dynamic Web Project）



设置项目名称

选择动态Web
模块的版本



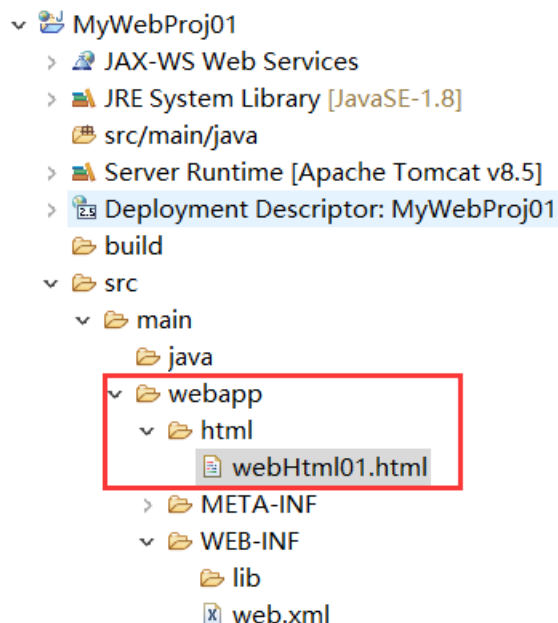
Eclipse中配置Tomcat

□ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– 创建页面

(1)在Webapp目录下新建文件夹，并在改文件夹内部创建一个html文件；

(2)输入一些html元素；



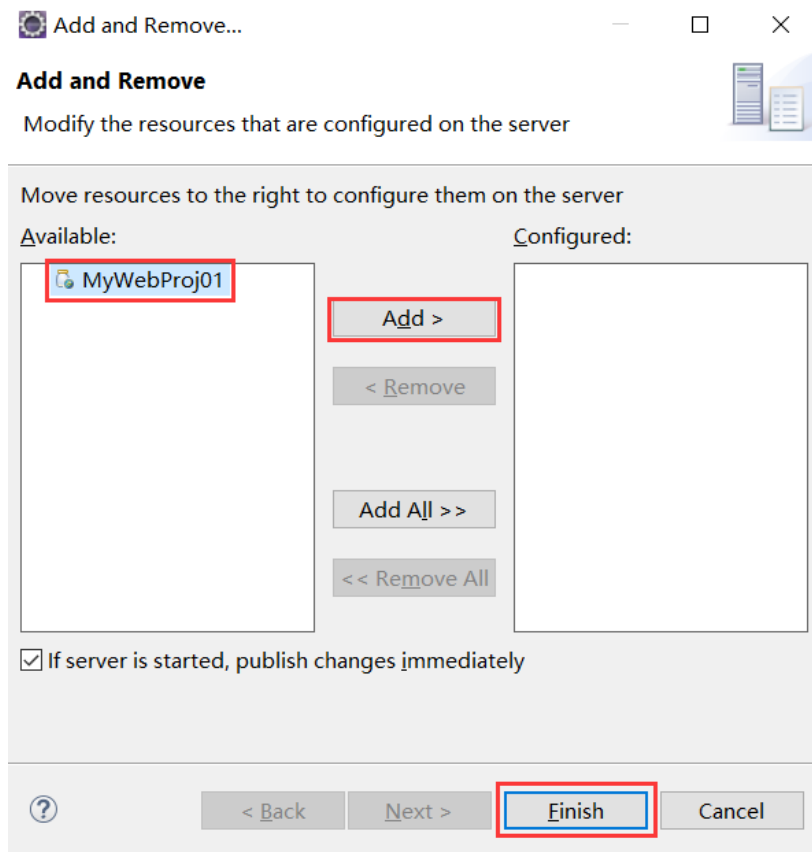
```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>第一个Web项目测试页面</h1>
</body>
</html>
```


Eclipse中配置Tomcat

❑ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– 发布项目，将该项目添加到Tomcat服务器中

在Servers窗口--右击Tomcat服务--Add and Remove

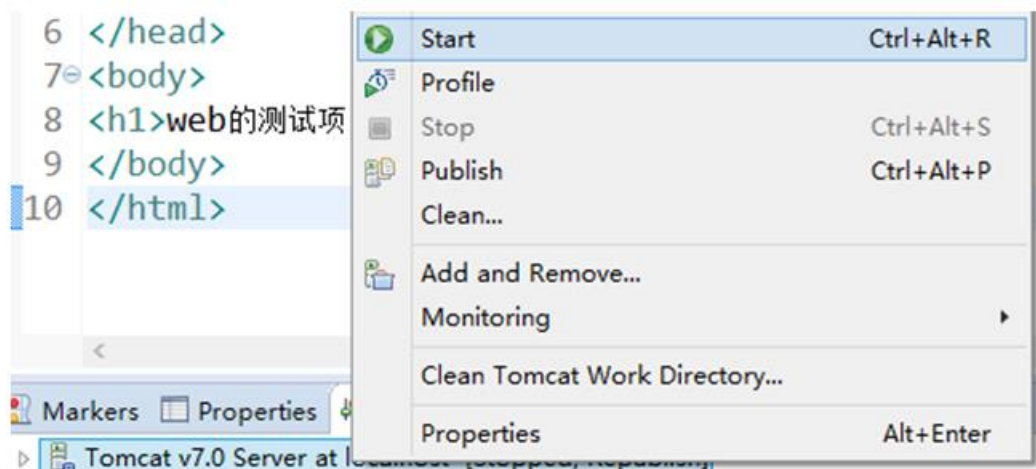


Eclipse中配置Tomcat

□ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

– 启动服务

在Servers窗口--右击Tomcat服务--Start



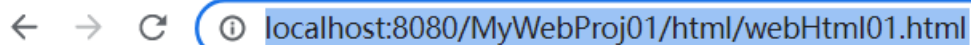
```
五月 19, 2022 9:13:10 下午 org.apache.catalina.startup.HostConfig deployDirectory
信息: 把web 应用程序部署到目录 [F:\software\professional software\tomcat\webapps\WebProj]
五月 19, 2022 9:13:10 下午 org.apache.catalina.startup.HostConfig deployDirectory
信息: Web应用程序目录[F:\software\professional software\tomcat\webapps\WebProj]的部署已在[17]毫秒内完成
五月 19, 2022 9:13:10 下午 org.apache.coyote.AbstractProtocol start
信息: 开始协议处理句柄["http-nio-8080"]
五月 19, 2022 9:13:10 下午 org.apache.catalina.startup.Catalina start
信息: Server startup in 654 ms
```

□ 在Eclipse中新建一个动态Web项目，并将其配置到Tomcat中

— 测试项目

在浏览器中输入：localhost:8080/项目名称/html文档目录名称/html文件名称

如：http://localhost:8080/MyWebProj01/html/webHtml01.html



← → ↻ ⓘ localhost:8080/MyWebProj01/html/webHtml01.html

第一个Web项目测试页面

成功!!!

- 1 JDBC的SQL注入漏洞
- 2 Eclipse中配置Tomcat
- 3 初识Servlet

□ 什么是Servlet?

Servlet其实就一个运行在web服务器上的小的Java程序，用于处理从web客户端发送的请求，并且对请求作出响应，使用Http协议。

- 运行在Web服务器上的小的Java程序；
- 处理请求并做出响应，将数据传回页面（动态网页）；
- 基于Http协议（超文本传输协议），该协议包含请求与响应两部分；
- Servlet是动态网页技术的基础

注：动态网页：页面html代码虽然没有变，但是显示的内容却是可以随着时间、环境或者数据库操作的结果而发生改变的；

不要将动态网页和页面内容是否有动画特效混为一谈（不是指JS动画）。

□ Http协议的请求与响应？

— Http协议请求部分

请求部分

```
POST /web_test/demo2.html HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
X-HttpWatch-RID: 20790-10010
Referer: http://localhost:8080/web_test/demo1.html
Accept-Language: zh-CN
User-Agent: Mozilla/5.0 (Windows NT 6.3; WOW64; Trident/7.0; rv:11.0) like
Gecko
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: localhost:8080
Content-Length: 19
DNT: 1
Connection: Keep-Alive
Cache-Control: no-cache
```

```
name=bbb&name=woman
```

请求行

请求头

请求体

□ Http协议的请求与响应？

– Http协议请求部分

● 请求行

■ 请求方式

◆ 请求方式有很多种，常用就两种GET和POST（默认GET方式）。

◆ GET和POST的区别？

- GET：请求的参数会显示到地址栏，大小有限制，没有请求体
- POST：请求的参数不会显示到地址栏（在请求体中）没有大小的限制。有请求体（请求参数）

■ 请求路径

■ 协议版本

● 请求头

■ 请求头的格式一般key, value键值对的形式。

■ 记住一些请求头：

◆ Referer：代表网页的来源。（防盗链）。

◆ User-Agent：获得客户端浏览器类型。

● 请求体

■ 只有POST请求方式才有请求体，内容是POST方式的请求参数。

□ Http协议的请求与响应？

— Http协议响应部分

响应部分内容

HTTP/1.1 200 OK

→ 响应行

Server: Apache-Coyote/1.1
Accept-Ranges: bytes
ETag: W/"151-1484111223984"
Last-Modified: Wed, 11 Jan 2017 05:07:03 GMT
Content-Type: text/html
Content-Length: 151
Date: Wed, 11 Jan 2017 05:45:53 GMT

→ 响应头

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Success 锈伙给锈?</h1>
</body>
</html>
```

→ 响应体

□ Http协议的请求与响应？

— Http协议响应部分

● 响应行

■ 协议版本

■ 状态码

◆200：代表响应成功

◆302：需要进行重定向操作

◆304：需要查找本地缓存

◆404：请求资源不存在

◆500：服务器内部错误

■ 状态码描述

● 响应头

■ 响应头的格式一般key, value键值对的形式。

■ 记住响应头：

◆Location：重定向的路径。

◆Refresh：定时刷新。

◆Content-Disposition：文件下载的时候使用。

● 响应体

■ 就是显示到浏览器上页面的代码。

初识Servlet

□ 创建第一个Servlet

具体步骤：(1)新建一个动态Web项目；

(2)在src下新建一个包，并新建一个Servlet；

(3)在Service()中输入：response.getWriter().println("Hello,Servlet!");

(4)启动Tomcat服务并访问Servlet。

在浏览器中输入：localhost:8080/项目名称/Servlet名称，观察结果！

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns:java.com/2002/04/web-app" xsi:schemaLocation="http://xmlns:java.com/2002/04/web-app http://xmlns:java.com/2002/04/web-app.xsd">
  <display-name>MyWebProj01</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <description></description>
    <display-name>MyServlet_1</display-name>
    <servlet-name>MyServlet_1</servlet-name>
    <servlet-class>it.hy.hj.servletDemo1.MyServlet_1</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>MyServlet_1</servlet-name>
    <url-pattern>/MyServlet_1</url-pattern>
  </servlet-mapping>
</web-app>
```

□ Servlet的执行流程

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns:
  <display-name>MyWebProj01</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <description></description>
    <display-name>MyServlet_1</display-name>
    <servlet-name>MyServlet_1</servlet-name>
    <servlet-class>it.hy.hj.servletDemo1.MyServlet_1</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>MyServlet_1</servlet-name>
    <url-pattern>/MyServlet_1</url-pattern>
  </servlet-mapping>
</web-app>
```

pattern, servlet-

□ Servlet的继承体系

(1)接口：Servlet；

(2)实现类：GenericServlet（与协议无关）；

HttpServlet（与协议相关，专门处理http协议的请求）。

(3)复写方法：复写HttpServlet中的doGet()和doPost()方法（HttpServlet中特有的方法），此时就不需复写Service()方法。

谢谢!

Thanks for your attention!